

CANNBot算子自动生成之RegBase

面向 Ascend 950 的 RegBase 矢量算子开发

课程目标

- 掌握使用 CANNBot 生成 RegBase 矢量算子的流程与关键方法

目录

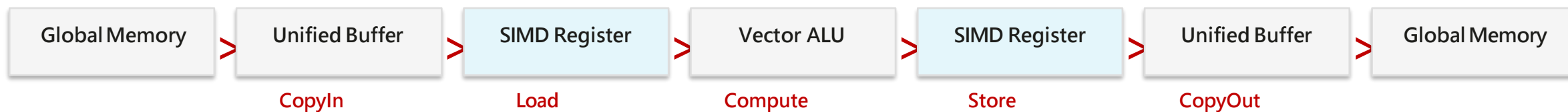
Part 1 RegBase 矢量编程

Part 2 CANNBot 如何支持 RegBase

Part 3 两个实操案例

RegBase 架构与数据流

- 什么是Regbase: Ascend 950 提供的一种更贴近新硬件平台的矢量能力写法



- 数据先从全局内存搬到 UB，再进入寄存器参与计算
- 核心计算尽量在寄存器里连续完成，减少中间结果反复写回 UB
- 代码里重点关注两件事：RegTensor 表示数据，MaskReg 处理尾块

RegBase 优化思路与代际对比

Ascend 910 MemBase 写法

- LocalTensor 上串联多个 API
- 中间结果占用临时 UB
- 每个节点可能读写一次 UB

Ascend 950 RegBase 写法

- VF 融合多个计算节点
- 中间结果直接在寄存器间交换
- 减少 VF 发射和 UB 读写开销

优化抓手：VF 融合、对齐搬运、PostUpdate、HardwareLoop；收益需要通过 profiling 验证。

MemBase 与 RegBase 实际样例对比

MemBase : LocalTensor 计算链

```
LocalTensor<float> xLocal = inQueueX.DeQue<float>();
LocalTensor<float> yLocal =
outQueueY.AllocTensor<float>();
LocalTensor<float> tmpLocal = tmpBuf.Get<float>();

Exp(tmpLocal, xLocal, n);
Adds(tmpLocal, tmpLocal, 1.0f, n);
Div(yLocal, xLocal, tmpLocal, n);

outQueueY.EnQue<float>(yLocal);
inQueueX.FreeTensor(xLocal);
```

RegBase : VF 内 RegTensor 计算链

```
__VEC_SCOPE
{
    RegTensor<float> vin, vtmp, vout;
    MaskReg mask = UpdateMask<float>(size);

    DataCopy<float, LoadDist::DIST_NORM>(vin, inAddr);
    Exp(vtmp, vin, mask);
    Adds(vtmp, vtmp, 1.0f, mask);
    Div(vout, vin, vtmp, mask);
    DataCopy<float, StoreDist::DIST_NORM_B32>(outAddr,
vout, mask);
}
```

- MemBase 的核心计算以UB 上的 LocalTensor 为操作对象
- RegBase 的核心计算进入 VF，以 RegTensor / MaskReg 表达寄存器状态和有效 lanes

目录

Part 1 RegBase 矢量编程

Part 2 CANNBot 如何支持 RegBase

Part 3 两个实操案例

CANNBot 支持 RegBase 的能力补齐

- 知识: RegBase 架构、API、reference、常见约束
- 生成: Host /UB /VF 边界和 Reg API 使用边界
- 验证: 编译运行、Golden /tolerance、Review checklist
- 反馈: 根据错误日志和审查结论回到设计与代码修复

CANNBot 如何支持 RegBase

Route	Reference	Design	Verify
识别 Ascend 950 RegBase 任务场景	读取 RegBase 矢量材料 和真实算子 reference	恢复 Host / UB / VF 边 界与生成结构	build / run / Golden / Review / profiling 闭环

知识注入不是把材料全文塞进上下文，而是抽取当前任务需要的架构、API、pattern 和 checklist。

完整 CANNBot 开发流程

1

环境检查

init project -> verify environment

平台约束: Ascend 950

2.5

串讲

Developer 质疑, Architect 回应

审视边界: Host / UB / VF

4

审查

Reviewer 独立构建和精度验证

验证依据: whitelist / Golden / tolerance

6-7

验收汇报

性能采集、结论和遗留问题

2

设计

Architect 输出 DESIGN.md / PLAN.md

知识进入: 架构、API、reference、checklist

3

开发

基于模板增量生成和编译

生成约束: RegTensor / MaskReg / Reg API

5

修复循环

最多三轮修复与复审

反馈回流: 错误日志 -> 设计与代码修复

CANNBot 与开发者的分工

开发者

- 明确需求、输入输出和目标平台
- 确认语义、边界条件和验收口径
- 审阅设计、代码和验证结论

CANNBot

- 识别 RegBase 场景并注入相关知识
- 生成 Host /UB /VF 结构和核心实现
- 组织编译、精度、Review 和修复闭环

实操中，开发者负责判断和收口，CANNBot 负责把流程、代码和验证材料连续产出。

目录

Part 1 RegBase 矢量编程

Part 2 CANNBot 如何支持 RegBase

Part 3 两个实操案例

实操一：从零开发 RegBase GELU 类算子

输入

需求、shape、dtype、Ascend 950 目标平台

知识

RegBase 矢量编程模型 + GELU RegBase reference

生成

RegBase Host /UB /VF 结构

验收

编译、运行、Golden / tolerance、Review checklist

资源管理器

WORKSPACE

- code
- dev_skills
 - ops
 - skills
 - ascendc-code-review
 - ascendc-direct-invoke-template
 - ascendc-docs-gen
 - ascendc-docs-search
 - ascendc-env-check
 - ascendc-npu-arch
 - ascendc-performance-analysis
 - ascendc-performance-best-practices
 - ascendc-performance-optimization
 - ascendc-precision-debug
 - ascendc-regbase-best-practice
 - ascendc-registry-invoke-template
 - ascendc-registry-invoke-to-direct-invoke
 - ascendc-runtime-debug
 - ascendc-st-design
 - ascendc-task-focus
 - ascendc-tiling-design
 - references
 - SKILL.md
 - ascendc-ut-develop
 - ascendc-whitebox-design
 - cann-env-setup
 - cannbot-skill-review
 - export-session
 - ops-precision-standard
 - ops-profiling
 - ops-simulator
 - pypto-api-explore
 - pypto-golden-generate

大纲

时间线

终端

opencode

帮我生成支持Ascend950芯片的gelu算子，使用regbase开发方式

Build · GLM-5-Turbo Zhipu AI Coding Plan

tab agents ctrl+p commands

Tip Drag and drop images or PDFs into the terminal to add them as context

/mnt/workspace/code/dev_skills/plugins-official/ops-direct-invoke:main 1.14.48

问题 输出 调试控制台 终端 异常检测结果 性能调优结果 DumpTensor数据

实操二：MemBase 到 RegBase 的转换

读源实现

已有 Ascend 910 GELU 的 LocalTensor 计算链

提取语义

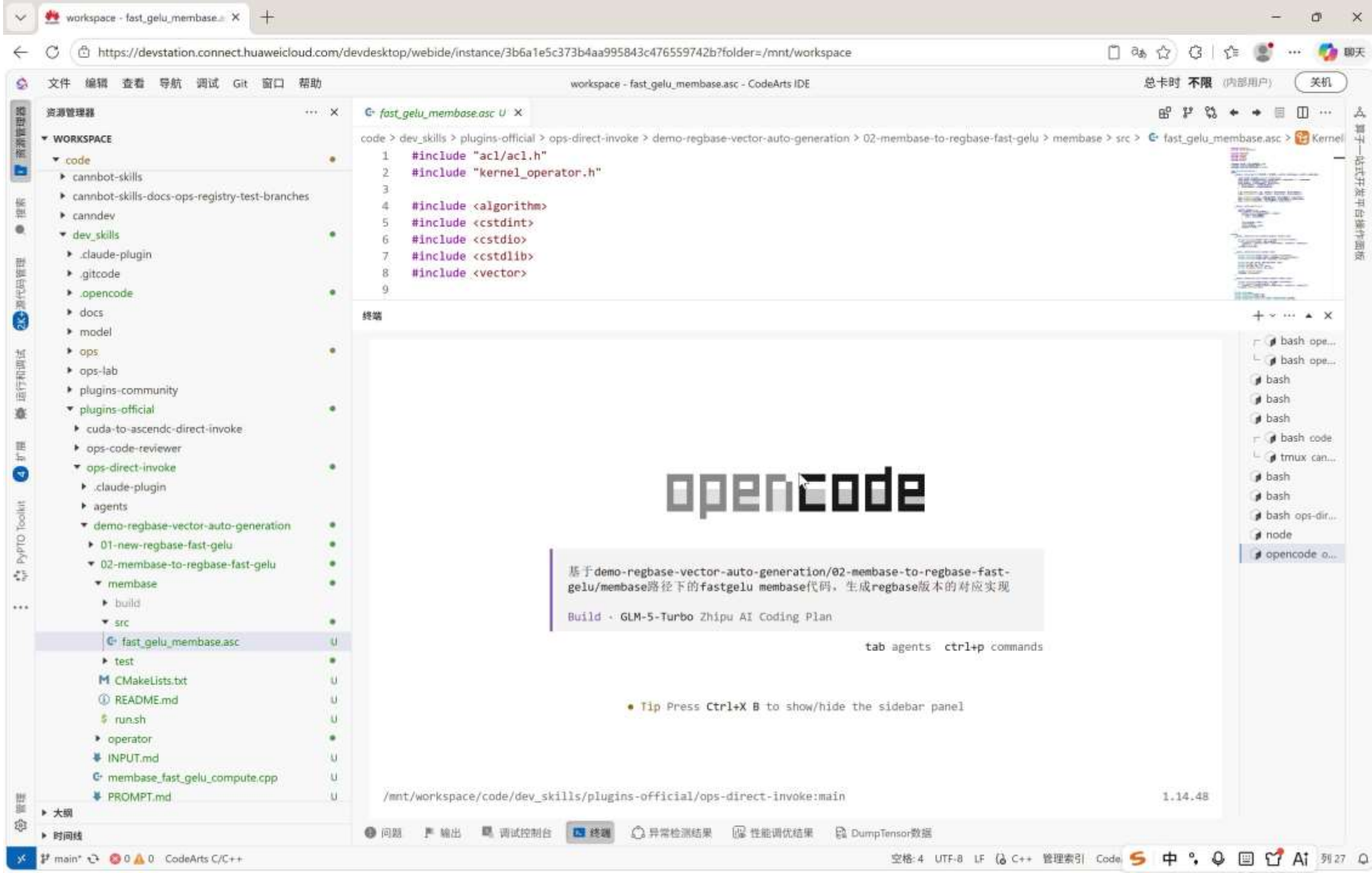
保留 GELU 语义和 CopyIn / Compute / CopyOut 数据流

目标转换

核心计算改写为 RegTensor + MaskReg + RegBase API

方法总结

新开发看目标结构，存量转换看复用边界，最后都用验证闭环收口



Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN